

第 16 章 | 循環神經網路：RNN 與序列建模

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家
教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線：序列資料的實務價值

在進入深度學習的高階領域時，我們必須重新審視「資料」的本質。傳統的監督式學習演算法（如支持向量機或隨機森林）通常建立在獨立同分布 (Independent and Identically Distributed, IID) 的假設上。這意味著訓練樣本之間是相互獨立的，且來自相同的分布。在處理 IID 資料（如 Iris 鳶尾花資料集）時，輸入資料的順序對模型而言毫無意義——測量花的順序並不會改變分類的準確性。

然而，當我們面對現實世界的複雜任務，如預測股價或翻譯句子時，資料的「順序」往往承載了比資料本身更多的實務價值。序列資料 (Sequential Data) 的元素之間並非獨立，而是存在強烈的前後依賴關係。如果將股價變動順序隨機化，我們將徹底喪失預測趨勢的能力。因此，從傳統機器學習跨入深度學習的關鍵轉折，在於模型是否具備處理這些「依賴關係」的能力。

IID 資料與序列資料的比較分析

特性	獨立同分布資料 (IID)	序列資料 (Sequential Data)
元素關係	相互獨立，互不影響	彼此依賴，存在順序性
順序重要性	無關緊要（順序不影響結果）	至關重要（順序定義含義）
典型假設	樣本 $x(1), x(2), \dots$ 順序可調換	順序 $\langle x(1), x(2), \dots, x(T) \rangle$ 不可調換
典型範例	鳶尾花分類、圖像靜態識別	股票預測、語音辨識、時間序列資料 (Time-series)

在理解序列資料對順序的嚴苛要求後，我們將探討如何利用 RNN 的架構來捕捉這些時間上的依賴性。

2. RNN 核心架構與運作邏輯

循環神經網路 (Recurrent Neural Networks, RNN) 的設計核心在於透過「循環」建立模型的記憶能力。與傳統的前饋神經網路 (FFNN) 不同，RNN 引入了循環邊 (Recurrent edge)，使資訊能從前一個時間步長流向當前步驟。

核心架構解析：展開與權重共享

RNN 的運作可以透過展開 (Unfolding) 機制來理解。在每一個時間步長 t ，隱藏層不僅接收當前的輸入 $x(t)$ ，還會結合來自前一刻的隱藏層狀態 $h(t-1)$ 。在數學表達上，隱藏層的預激活值 (Preactivation) $zh(t)$ 與輸出 $h(t)$ 計算如下：

$$zh(t) = W_x h_x(t) + W_{hh} h(t-1) + b_{hh}(t) = \text{phih}(zh(t))$$

其中 $W_x h$ 是輸入權重， W_{hh} 是循環權重，而 phih 通常為 $\tanh$ 激活函數。值得注意的是，這些權重在所有時間步長上都是相同的，這種權重共享 (Weight Sharing) 機制極大化了參數效率，並使模型能處理任意長度的序列。

建模類別歸納

除了標準的隱藏層循環 (Hidden-to-hidden recurrence) 外，學界也存在輸出到隱藏層 (Output-to-hidden) 或輸出到輸出 (Output-to-output) 的變體。根據輸入與輸出的對應關係，我們歸納出三種實務場景：

多對一 (Many-to-One)：輸入為序列，輸出為單一向量（如：電影評論的情感分析）。

一對多 (One-to-Many)：輸入為固定格式，輸出為序列（如：圖像描述 Image Captioning）。

多對多 (Many-to-Many)：

同步型 (Synchronized)：每一幀輸入對應一幀輸出（如：影片標籤分類）。

延遲型 (Delayed)：讀取完完整序列後再生成輸出（如：機器翻譯）。

雖然 RNN 架構設計精巧，但在處理長期記憶時，它面臨了重大的數學挑戰。

3. 長期依賴挑戰與 LSTM 的崛起

在訓練 RNN 時，我們使用 隨時間反向傳播 (Backpropagation Through Time, BPTT)。然而，隨著序列長度 T 的增加，權重矩陣 W_{hh} 的反覆乘積會導致嚴重的梯度問題。

問題診斷：梯度的消失與爆炸

在計算梯度時，會涉及項 $\partial h(k) \partial h(t) = \prod_{i=k+1}^t \partial h(i-1) \partial h(i)$ 。這意味著權重會以指數級影響梯度：

梯度消失 (Vanishing Gradient)：當權重 $|w| < 1$ ，長距離依賴資訊會隨時間衰減至零，導致模型「遺忘」早期關鍵資訊。

梯度爆炸 (Exploding Gradient)：當權重 $|w| > 1$ ，梯度會劇烈膨脹，導致訓練崩潰。常用的對策是梯度裁剪 (Gradient Clipping) 或截斷隨時間反向傳播 (Truncated BPTT, TBPTT)。

LSTM 的門控機制

為了從根本解決問題，長短期記憶網路 (Long Short-Term Memory, LSTM) 引入了 細胞狀態 (Cell state, $C(t)$)。它作為「資訊高速公路」，透過門控結構精準控制資訊流。其數學定義如下：

遺忘門 (Forget gate)： $f_t = \sigma(W_x f_x(t) + W_h f_h(t-1) + b_f)$

輸入門 (Input gate)： $i_t = \sigma(W_x i_x(t) + W_h i_h(t-1) + b_i)$

候補細胞狀態： $\tilde{C}_t = \tanh(W_x c_x(t) + W_h c_h(t-1) + b_c)$

更新細胞狀態： $C(t) = (C(t-1) \odot f_t) \oplus (i_t \odot \tilde{C}_t)$

輸出門 (Output gate)： $o_t = \sigma(W_x o_x(t) + W_h o_h(t-1) + b_o)$

最終隱藏狀態： $h(t) = o_t \odot \tanh(C(t))$

此外，門控循環單元 (Gated Recurrent Unit, GRU) 則是 LSTM 的精簡化變體，運算效率更高，在特定場景下效能相當。

掌握了理論後，我們將進入第一個實戰專案：利用 LSTM 進行電影評論的情感分析。

4. 實戰專案一：IMDb 電影評論情感分析 (Many-to-One)

本專案旨在將非結構化文本轉化為結構化情緒標籤（0為負面，1為正面）。

資料預處理與嵌入層 (Embedding)

Tokenization & Encoding：將評論切割為單詞，並映射為整數索引。

Unknown Words：若詞彙量為 q ，未見過的詞將被映射為 $q+1$ ；索引 0 則保留給 Padding。

Padded Batch：使用 `padded_batch()` 處理變長序列，確保同一個 batch 內維度一致。

嵌入層是此專案的技術核心。相對於維度極高且稀疏的獨熱編碼 (One-hot)，嵌入層是一個可訓練的權重矩陣（即 Lookup Table）。它具備兩大優勢：

降維：將數萬維的字彙空間壓縮至低維連續空間（如 20 維）。

語義表徵：透過訓練，語義相近的詞在空間中的歐氏距離會逐漸縮小。

模型建構與訓練

我們使用 TensorFlow Keras 建構雙向 LSTM (Bidirectional LSTM)，並設定實務 hyperparameters。

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=vocab_size, output_dim=20),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
    metrics=['accuracy']
)
```

情感分析是序列資料的一種簡化應用，接下來我們挑戰更複雜的任務：字符級的文本生成。

5. 實戰專案二：字符級語言模型與文本生成 (Many-to-Many)

在字符級建模中，我們不再處理單詞，而是學習預測下一個字符。此過程稱為 自動回歸 (Auto-regression)，即將模型生成的輸出作為下一步的輸入。

資料重構邏輯

我們採用 Jules Verne 的《神祕島》文本，將其切割成長度為 41 的片段。

輸入 x ：片段的前 40 個字符（索引 0 至 39）。

目標 y ：向右平移一格的 40 個字符（索引 1 至 40）。

取樣策略與溫度控制

在推理階段，我們不直接選取最高機率的字符（Greedy search），而是從 Logits 分布中隨機取樣。此處引入縮放因子 (α)，它是物理學中溫度的「倒數」：

$\alpha=2.0$ (低溫)：放大 Logits 差異，生成內容更穩健、符合語法，但缺乏新意。

$\alpha=0.5$ (高溫)：使分布趨於均勻，增加隨機性與創造力，但也更容易產生亂碼。

技術提示：訓練不收斂通常是因為序列過長或梯度爆炸，可嘗試使用梯度裁剪 (Gradient Clipping)。

雖然 RNN 與 LSTM 在序列處理上表現優異，但新一代的架構已經打破了循環的限制。

6. 前瞻視野：Transformer 與自注意力機制

2017 年提出的 Transformer 架構改變了 NLP 的建模方式。它不使用循環結構，改用 自注意力機制 (Self-attention mechanism) 實現全局建模。

核心觀念：Q、K、V 的協奏

自注意力機制透過三個向量計算元素間的關係：

查詢 (Query, Q)：「我正在尋找什麼樣的資訊？」

鍵 (Key, K)：「我具備什麼樣的標籤？」

值 (Value, V)：「我實際承載的內容是什麼？」

縮放點積注意力 (Scaled Dot-Product Attention)

計算公式為： $\text{Attention}(Q, K, V) = \text{softmax}((QK^T) / \sqrt{d_k}) V$ 。其中 $\sqrt{d_k}$ 用來縮放點積，避免數值過大讓 softmax 過度集中。

本課程至此已完成從基礎序列觀念到現代尖端架構的全面覆蓋。

7. 動手練習與自我檢查表

自我檢查清單

- ☐ 我能解釋為什麼序列資料不符合 IID 假設嗎？
- ☐ 我理解 Whh 在 BPTT 中如何導致梯度消失嗎？
- ☐ 我能寫出 LSTM 遺忘門、輸入門與細胞狀態的更新公式嗎？
- ☐ 我理解 Embedding Layer 作為「可訓練查找表」的優勢嗎？
- ☐ 我知道縮放因子 α 與生成文本隨機性的關係嗎？

動手練習題

觀念題：為什麼處理 500 字以上的長文本時，SimpleRNN 通常無效？請從權重 $wt-k$ 的數學特性解釋。

實作題：在文本生成實驗中，若將 `scale_factor` (α) 設為 0.1，預期產生的文本會偏向「高度重複」還是「混亂無序」？為什麼？

關鍵總結 (Takeaways)

順序很重要：序列建模的核心在於捕捉時間軸上的非獨立性依賴。

門控是核心：LSTM 透過精確的門控數學機制，穩定了長期資訊的傳遞路徑。

並行是趨勢：Transformer 透過自注意力機制解決了 RNN 的序列依賴瓶頸，是現代大型語言模型的基石。